

CampusEats — SOAP Partner Integration

Payment Gateway Edge: SecurePay

Aman Bali (20251651015)

Bhupesh Kumar (20251651032)

August 2026

1. Context

Partner: SecurePay — an external payment gateway. **Operation integrated:** charge, called by CampusEats' own **Payment Service** (Section 3.5 of the Assignment 2 contract, `makePayment`) to authorize and capture a student's payment for a placed order.

This edge is SOAP while the rest of CampusEats is REST for three reasons. First, a payment charge is a **financial transaction with legal and audit weight**, so it needs a strongly-typed, machine-checkable contract (the WSDL) rather than a loosely-typed JSON body — a schema mismatch here should fail fast at design time, not silently in production. Second, the call carries **message-level security**: merchant credentials travel in a signed/encrypted SOAP header (Credentials), separable from the transport, which matters if the message is ever relayed through an intermediary rather than going over a single TLS hop. Third, payment gateways in practice standardise on WSDL/SOAP with well-defined `soap:Fault` semantics for declines, timeouts, and reversals, so integrating on the partner's own terms avoids building and maintaining a translation layer for a partner that will not offer a REST alternative.

2. Partner Contract (WSDL)

The full contract is submitted as `partner.wsdl`. It defines all six required elements:

Element	What it contains
types	Credentials (header), ChargeRequest, ChargeResponse, ChargeFaultDetail, plus the CaptureMode / ChargeStatus enumerations
message	ChargeHeaderMessage, ChargeRequestMessage, ChargeResponseMessage, ChargeFaultMessage
portType	SecurePayGatewayPortType with one operation, charge (input / output / fault)

Element	What it contains
binding	SecurePayGatewayBinding — SOAP 1.1 over HTTP, document/literal style
service / port	SecurePayGatewayService / SecurePayGatewayPort, endpoint <code>https://api.securepay-partner.com/soap/v1/gateway</code>

The charge operation takes a merchant order reference, amount, currency, a tokenised card, and a capture mode, and returns a transaction id, status, and authorisation code — or a `ChargeFault` carrying a partner fault code such as `card_declined`.

3. SOAP Messages

Three envelopes are submitted, hand-written element-for-element against `partner.wsdl`:

- **soap-request.xml** — the Credentials header (`apiKey`, `merchantId`) plus a `ChargeRequest` body for order `ORD-2026-08-004417`.
- **soap-response.xml** — the matching success `ChargeResponse`: a transaction id, status `CAPTURED`, an auth code, and a timestamp.
- **soap-fault.xml** — a `soap:Fault` for a realistic decline, with a `ChargeFaultDetail` in `<detail>` carrying the partner's own fault code `card_declined`.

4. HTTP Binding

The request in `soap-request.xml` is carried as the following HTTP POST, per the `soap:address` and `soapAction` declared in the binding:

```
POST /soap/v1/gateway HTTP/1.1
Host: api.securepay-partner.com
Content-Type: text/xml; charset=utf-8
Content-Length: 812
SOAPAction: "http://securepay.example.com/gateway/v1/charge"
```

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
                xmlns:sptypes="http://securepay.example.com/gateway/v1/types">
  <soap:Header>
    <sptypes:Credentials>
      <sptypes:apiKey>sp_live_9f2b7a6cle4d</sptypes:apiKey>
      <sptypes:merchantId>CAMPUSEATS_MERCH_0142</sptypes:merchantId>
    </sptypes:Credentials>
  </soap:Header>
  <soap:Body>
    <sptypes:ChargeRequest>
      <sptypes:merchantOrderRef>ORD-2026-08-004417</sptypes:merchantOrderRef>
      <sptypes:amount>287.50</sptypes:amount>
```

```

        <sptypes:currency>INR</sptypes:currency>
        <sptypes:cardToken>tok_4242c9a1b7</sptypes:cardToken>
        <sptypes:captureMode>AUT0</sptypes:captureMode>
    </sptypes:ChargeRequest>
</soap:Body>
</soap:Envelope>

```

The endpoint (Host + path) is taken directly from soap:address location in partner.wsdl; the SOAPAction header value is taken directly from soap:operation soapAction on the charge operation.

5. Discovery (Modern Form)

CampusEats would obtain this WSDL in one of two ways in practice: (a) a **direct URL** published by SecurePay in their developer portal (e.g. `https://developer.securepay-partner.com/wsdl/v1/gateway.wsdl`), fetched once at integration time and vendored into the CampusEats repo (as `partner.wsdl` here); or (b) a **lookup against an internal API/service catalogue** that CampusEats' platform team maintains for all external partners, which is the modern replacement for a UDDI registry — no UDDI server is run for this assignment. The catalogue entry below plays the tModel role: it names the business, the service, the live endpoint, and points at the WSDL that describes how to speak to it.

Registry / catalogue entry

Field	Value
Business	SecurePay Inc. (payment gateway partner)
Service	SecurePay Gateway — Charge API v1
Endpoint	<code>https://api.securepay-partner.com/soap/v1/gateway</code>
WSDL (tModel pointer)	<code>https://developer.securepay-partner.com/wsdl/v1/gateway.wsdl</code>
Binding	SOAP 1.1 / HTTP, document/literal
Owning CampusEats service	Payment Service (makePayment)
Contact	<code>partner-integrations@securepay-partner.com</code>

6. Fault Mapping

The partner's fault vocabulary must never leak into CampusEats' own contract. The Payment Service is the only place that sees `partner.wsdl`'s `ChargeFault`; every other CampusEats service only ever sees the errors already promised in the Assignment 2 contract (`PaymentDeclined`, `GatewayTimeout`, `PaymentNotFound`, and — one level up — `placeOrder`'s `PaymentFailed`). The Payment Service's `makePayment` implementation catches the SOAP fault and translates it internally:

SecurePay soap:Fault (ChargeFaultDetail.faultCode)	CampusEats contract error (Section 3.5 / 4)
card_declined	PaymentDeclined (from makePayment) → surfaces to placeOrder as PaymentFailed
gateway_unavailable / SOAP transport timeout	GatewayTimeout (from makePayment) → surfaces to placeOrder as PaymentFailed
invalid_amount	InvalidAmount (from makePayment)
any unrecognised partner fault code	GatewayTimeout (safe default; logged with the raw partnerReference for support, never returned to the caller)

A caller of makePayment — including placeOrder — never sees ChargeFaultDetail, faultCode strings, or the SecurePay envelope at all; it only ever sees the named errors from CampusEats' own contract, exactly as required by the Assignment 2 boundary rule.